Hand and Signature Analysis with Computer Vision

An experimental study using YOLOv8n, MediaPipe, and OpenCV

Sana Gholipour Bashiri

Abstract

This report consolidates two computer-vision projects: hand detection and landmark visualization, and exploratory signature-region detection. For the hand task, a small annotated dataset was organized into training and validation subsets, and a pretrained YOLOv8n model was fine-tuned in two 30-epoch runs. The recorded split contained ten training images and three validation images. Although the saved best-checkpoint evaluation reported high average precision, the subsequent video experiment produced no detections across 174 frames at a confidence threshold of 0.30. A separate MediaPipe implementation was developed to draw hand landmarks on still images and video frames. The original hand report describes successful video processing; however, the supplied notebook retains dependency errors rather than a completed demonstration. For the signature task, contrast enhancement, Otsu thresholding, and geometric contour filtering were combined with an exploratory pretrained YOLOv8n branch and TensorBoard logging. The saved YOLO output contains zero boxes, and an embedded TensorBoard scalar records zero accepted contour candidates. The work demonstrates data preparation, model training, image processing, visualization, and experimental logging while identifying the need for broader data, task-specific labels, and independent evaluation.

Keywords: hand detection; hand landmarks; signature regions; YOLOv8n; MediaPipe; OpenCV; TensorBoard.

1. Introduction and objectives

The two projects investigate how learned detectors and conventional image-processing methods can be applied to different visual targets. The hand project progresses from manually annotated object detection to a landmark-based alternative. The signature project examines whether foreground contours and a general pretrained detector can identify candidate signature regions in a photographed document. Signature-region detection is the intended task; writer identification and signature authenticity verification are outside the implemented scope.

The combined contribution is a documented experimental workflow: preparing hand annotations, training a custom detector, testing video inference, implementing landmark overlays, constructing a contour-based image pipeline, and recording outputs for inspection. This report describes what was implemented and what the retained evidence supports, including unsuccessful experiments.

Repository naming: Github-Yolov11 is the requested collection name. Every YOLO model loaded by the supplied code is YOLOv8n (*yolov8n.pt* or its fine-tuned checkpoint). No YOLO11 implementation or experiment is present.

2. Study design and source evidence

The study is organized into two application tasks with three implementation paths. The available materials comprise two original project reports and three Python notebooks. Both original reports are predominantly in Persian. This English synthesis uses their methodological descriptions together with notebook source cells, saved console outputs, and embedded TensorBoard resources.

2.1 Experimental workflow

HAND / YOLOv8n	HAND / MEDIAPIPE	SIGNATURE / TWO BRANCHES
Annotated image archive	Still image or video	Photographed document
JPG-label pairing; 10 / 3 split	BGR to RGB conversion	Grayscale + CLAHE
Two 30-epoch training runs	Hands inference	Otsu contours | YOLOv8n
Best checkpoint; video test	Landmarks + connections	Geometry filter | book proxy
Saved inference log	Image display / video writing	Counts + image in TensorBoard

Figure 1. Implemented paths reconstructed from the notebooks. The YOLO branches use YOLOv8n. The diagram describes program flow and is not a performance result.

2.2 Evidence and interpretation

Three evidence levels are kept distinct. Source code establishes which operations were implemented. Retained outputs establish what was recorded during particular runs. Narrative statements in the original reports establish the author's description of the work but do not replace missing execution artifacts. No new model training, inference, or accuracy measurement was performed for this consolidation.

The notebooks were copied byte for byte and given descriptive filenames. Their outputs, comments, metadata, repeated training cells, and environment-specific commands were preserved. A SHA-256 manifest in the Code folder records the original-to-new mapping.

2.3 Boundaries of the evidence

The supplied folder does not contain the hand dataset archive, annotation files, original hand image, input videos, trained weights, or standalone TensorBoard event logs. The signature notebook does retain a cached image and one fetched scalar inside its Colab metadata. These cached resources allow limited recovery of the recorded signature experiment, but do not constitute a complete reproducibility package.

3. Custom hand detection with YOLOv8n

3.1 Annotation and dataset organization

The original hand report states that manual annotations were prepared with makesense.ai before trying YOLO. In the training notebook, hands_dataset.zip is extracted under /content/hand_dataset. The code lists the extracted files, collects JPEG images with the pattern *.jpg, and pairs each selected image with a same-stem text label. It then creates images/train, images/val, labels/train, and labels/val directories and moves the paired files into these directories.

The intended split is 80% training and 20% validation, using train_test_split with random_state=42. The recorded dataset scans show ten training images and three validation images. The extracted listing includes hand1.jpeg, but the preparation code selects only *.jpg; that file is therefore excluded by the implemented collection step. A data.yaml file specifies a single class, hand, with the two image directories as the training and validation paths.

3.2 Training configuration

Setting	Recorded value
Initial checkpoint	yolov8n.pt
Target classes	1: hand
Training / validation images	10 / 3
Epochs	30 in each of two training cells
Image size / batch size	640 / 16, as recorded in the trainer log
Optimizer selected automatically	AdamW; initial learning rate 0.002
Execution device	CPU, as recorded in the notebook output
Run directories	runs/detect/train and runs/detect/train2
Video confidence threshold	0.30

Table 1. Hand-detector configuration reconstructed from the source and trainer logs.

The two adjacent training cells each reload the pretrained YOLOv8n checkpoint and call model.train for 30 epochs. They are separate training invocations rather than one continuous 60-epoch run. Both retained runs show the same printed validation trajectories and summary metrics. The notebook subsequently loads runs/detect/train/weights/best.pt from the first run for inference.

3.3 Video inference

The model is applied to hand_video.mp4 with save=True and conf=0.3. The intended annotated output is saved under runs/detect/predict. The recorded output reports 174 processed frames, each marked no detections. The final diagnostic printout shows an empty box tensor for the first result and a class dictionary containing hand. The notebook's video display cell contains a path string in its HTML representation rather than recoverable video bytes.

4. Hand-detector results and interpretation

Table 2 separates the metrics printed at the last training epoch from the later best-checkpoint validation. Values are reproduced as logged; both training cells report the same values to the shown precision. The validation set contains only three images and three labeled instances.

Evaluation stage	Precision	Recall	mAP@.50	mAP@.50:.95
Epoch 30	0.00222	0.667	0.409	0.379
Best checkpoint	0.00333	1.000	0.995	0.962

Table 2. Retained hand-validation summaries. These are validation metrics, not independent test-set results. The best-checkpoint row uses the rounded console values; its stored metrics dictionary gives mAP@.50:.95 = 0.9620374.

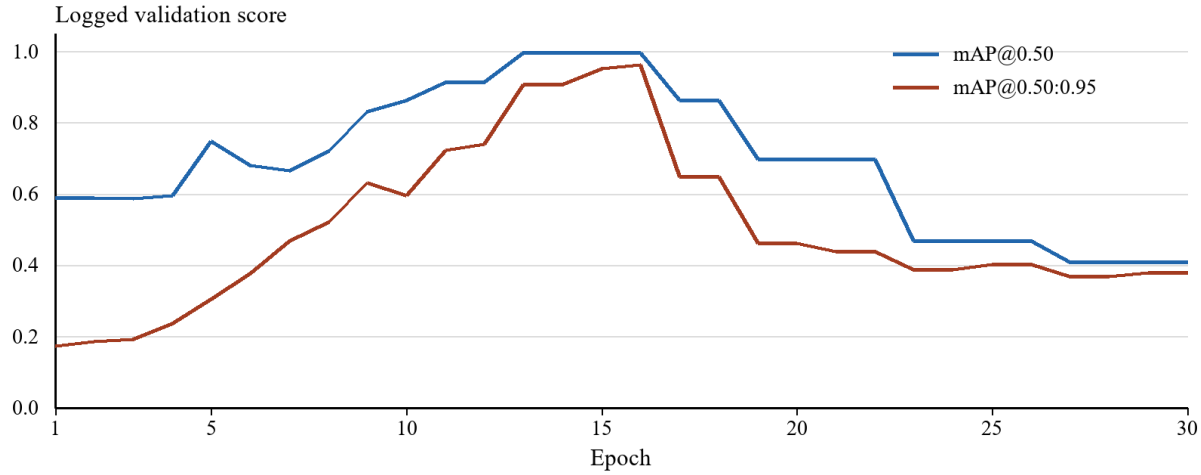


Figure 2. Validation mAP across the 30 logged epochs, reconstructed from the first training cell. The second cell has the same printed trajectory. The separate best-checkpoint re-evaluation is excluded from the epoch curves.

4.1 Relationship between validation and video behavior

High average precision in the best-checkpoint summary coexists with very low printed precision and a video run with no detections at the selected confidence threshold. These values are retained without attempting to reconcile them by inventing a new measurement. Average precision summarizes ranked detections across score thresholds, whereas a video inference run uses a selected cutoff. Nevertheless, the small validation set and the absence of an independent labeled video test prevent a strong generalization claim.

Potential contributors include a narrow training sample, differences between training images and video frames, annotation quality, and confidence calibration. These are hypotheses for future investigation, not demonstrated causes. The saved outputs alone do not establish which factor caused the unsuccessful video result. No video recall, precision, or localization accuracy can be calculated without frame-level ground truth.

4.2 Timing evidence

The video log reports mean preprocessing, inference, and postprocessing times of 3.6, 160.8, and 0.7 milliseconds per frame, respectively. These are notebook-reported timings from one CPU run. They do not include all input/output and rendering overhead and should not be treated as a controlled real-time benchmark.

5. Hand-landmark visualization with MediaPipe

5.1 Motivation and implementation

After the YOLO attempts did not provide the expected hand detections, the original report describes switching to MediaPipe. The alternative notebook initializes mp.solutions.hands.Hands with max_num_hands=2 and min_detection_confidence=0.7. OpenCV supplies image reading, color conversion, video decoding, and output encoding. MediaPipe drawing utilities overlay detected landmarks and HAND_CONNECTIONS.

The still-image path reads /content/hand.jpg, verifies that it was loaded, converts its color order from BGR to RGB, and passes it to hands.process. Detected landmarks are drawn on the original image. The code first tries an OpenCV window and includes a Matplotlib fallback for display errors. The MediaPipe resource is closed at the end.

5.2 Video processing procedure

Stage	Implemented operation
Input validation	Check /content/videohand.mp4 and open VideoCapture
Writer preparation	Read frame width, height, and integer FPS; use mp4v encoding
Frame processing	Decode a frame, convert BGR to RGB, and run hands.process
Visualization	Draw each detected hand landmark set and its connections
Output	Write frames to output_video.mp4 and increment frame_count
Cleanup	Release capture and writer; close MediaPipe Hands
Colab download	Call google.colab.files.download for the generated video

Table 3. Stages of the implemented MediaPipe video workflow.

The loop checks for end-of-video conditions, invalid frames, color-conversion failures, and writing errors. Every readable frame is intended to be written, including frames without hand landmarks. The displayed frame_count therefore counts processed frames rather than successful hand detections.

5.3 What the retained material establishes

The original hand report states that hands were identified, their landmarks were displayed, and a processed video was saved. However, both MediaPipe execution cells in the supplied notebook retain ModuleNotFoundError: No module named mediapipe. The setup cell installs Ultralytics, not MediaPipe. The implementation is present, but this particular notebook snapshot does not independently verify a completed landmark-processing run.

No landmark accuracy measurements, hand-detection counts, output frames, or processed MediaPipe video are included in the supplied files. The appropriate result is therefore a documented implementation accompanied by the author-reported qualitative outcome. A repeatable evaluation would require the input media, a compatible MediaPipe installation, saved output examples, and manually checked landmarks or hand-presence labels.

6. Exploratory signature-region detection

6.1 Image preparation and contour candidates

The signature notebook reads /content/signa.jpg and verifies that the image exists and can be decoded. The image is converted to grayscale and processed with contrast-limited adaptive histogram equalization (CLAHE), using clipLimit=2.0 and tileGridSize=(8,8). Otsu's method then produces an inverted binary image, intended to separate darker writing from the lighter paper.

External contours are extracted with cv2.RETR_EXTERNAL and cv2.CHAIN_APPROX_SIMPLE. The code rejects contours with an area below 500 pixels. Bounding rectangles are also rejected if either width or height is below 50 pixels or above 300 pixels. Each accepted contour increments the candidate count and receives a blue rectangle with the label Possible Signature (Contour).

Component	Parameter or condition
Contrast enhancement	CLAHE: clip limit 2.0; 8 x 8 tile grid
Thresholding	THRESH_BINARY_INV + THRESH_OTSU
Contour retrieval	External contours; simple boundary approximation
Minimum contour area	500 square pixels
Accepted rectangle width	50 to 300 pixels, inclusive
Accepted rectangle height	50 to 300 pixels, inclusive
Contour annotation color	Blue in the output image

Table 4. Preprocessing and candidate-filtering parameters in the signature notebook.

These geometric rules define possible regions rather than a learned signature classifier. Their fixed pixel thresholds depend on image scale. Printed lines, connected handwriting, illumination, and the size of the signature may affect which components are retained. The code does not include a ground-truth comparison, connected-component grouping, or a resolution-normalized threshold. These factors are relevant limitations of the implemented heuristic.

6.2 Exploratory YOLO branch

A pretrained yolov8n.pt model is loaded. The code searches the model's class names for signature; the saved output states that this class is unavailable and selects book, with class index 73, as a fallback. The contrast-enhanced grayscale image is converted back to three channels and supplied to the model. Only boxes matching the selected class are counted and drawn in red.

Selecting book changes which existing category is accepted; it does not train the model to recognize signatures. Consequently, this branch is an exploratory proxy and cannot substantiate signature detection capability. No signature-specific YOLO fine-tuning, labeled signature dataset, or model comparison is implemented in the supplied notebook.

7. Signature results and experiment logging

7.1 Retained observations

Evidence	Recorded observation
YOLO inference console output	No detections; zero boxes; printed no signature
Cached TensorBoard contour scalar	0 candidates at step 0
Image-writing return value	True for output_image_signature.jpg
Embedded TensorBoard image	Photographed signature; no visible detection boxes

Table 5. Retained signature observations from console outputs and cached TensorBoard data.

The contour scalar and image were recovered from cached TensorBoard responses in the final cell's Colab metadata. Their wall times differ, so they are not treated as synchronized observations. The notebook's YOLO text independently reports zero boxes.

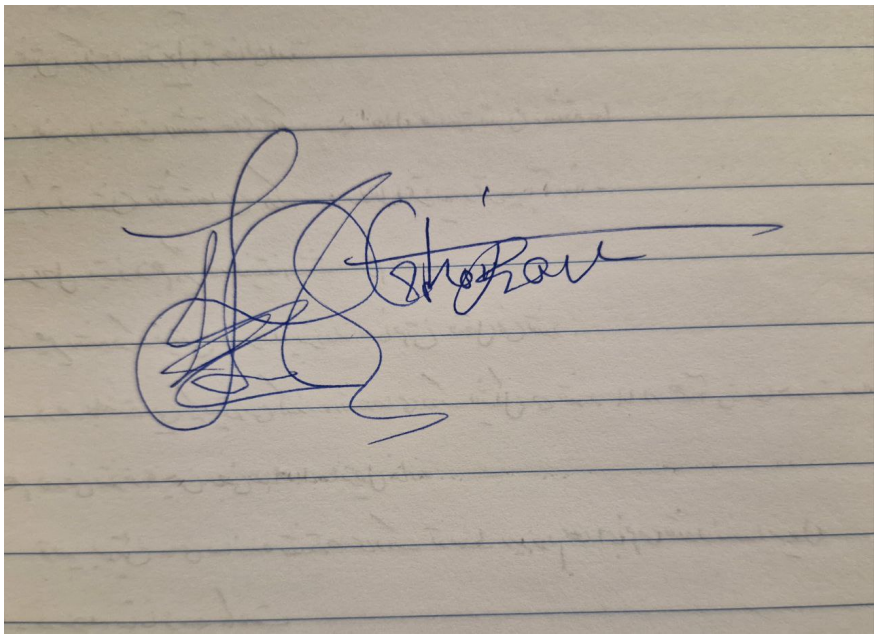


Figure 3. Original image recovered from the notebook's cached TensorBoard image resource. Its colors and content are unchanged. No detection overlays are visible. This is a retained output artifact, not a newly generated detection result.

7.2 Logging and output handling

SummaryWriter is configured with runs/signature_detection. The program writes contour and YOLO counts at step 0, adds an average-confidence scalar only if accepted YOLO boxes exist, and logs the annotated image after converting it to RGB and transposing it from HWC to CHW. It also saves output_image_signature.jpg and includes a Colab TensorBoard command. The supplied code does not explicitly call writer.flush() or writer.close().

Neither the original report's qualitative description nor these counts establish detection accuracy. There is one demonstrated document image, no labeled evaluation set, and no controlled comparison of the two branches. The results support completion of an exploratory processing and logging pipeline, with unsuccessful detections in the retained observations.

8. Reproducibility, discussion, and conclusion

8.1 Execution environment and inputs

The notebooks were written for an interactive Google Colab workflow and contain /content paths, installation magics, TensorBoard magics, and Colab-specific download functions. They remain .ipynb files because renaming them to .py would not convert those commands into ordinary Python. The notebook sources and recorded outputs are unchanged.

The hand-training log records Python 3.11.11, Ultralytics 8.3.105, and PyTorch 2.6.0+cu124 on CPU. The MediaPipe notebook setup output records Python 3.11.12 and Ultralytics 8.3.111; its MediaPipe cells fail to import the dependency. The signature setup output records Python 3.11.11, Ultralytics 8.3.111, and PyTorch 2.5.1+cu124 on CPU. These are historical output records, not a verified dependency lockfile.

Re-execution requires the missing hand dataset and media at the paths documented in Readme.tex. Image-label pairs must share a filename stem, the hand preparation cell moves files, and the current glob handles .jpg only. The MediaPipe workflow needs a compatible version that exposes mp.solutions.hands. The signature workflow needs OpenCV, NumPy, Ultralytics, PyTorch, and TensorBoard. The repository includes the recovered signature image as evidence, but the original signa.jpg is not independently supplied.

8.2 Discussion and next experiments

The principal methodological finding is that each method must be evaluated against its own target and evidence. A detector trained on a very small hand dataset cannot be judged by validation mAP alone when video inference produces no boxes. Landmark visualization requires a functioning environment and retained processed frames. A contour-size heuristic and a pretrained book class do not provide a validated signature-region detector.

Further hand experiments should expand and audit the annotations, retain a separate video-based test set, report threshold-dependent precision and recall, and compare the detector with landmark outputs on the same frames. Further signature experiments should use genuine signature-region labels, normalize scale-sensitive preprocessing, and compare contour grouping with a task-specific trained detector on labeled documents. These are proposed extensions and were not performed in the supplied projects.

8.3 Conclusion

The consolidated project documents practical work in annotation, dataset restructuring, transfer learning, video inference, landmark visualization, foreground extraction, and experiment logging. Its strongest supported outcome is the construction and examination of these workflows. The retained YOLO and contour observations expose important limitations, while the original hand report motivates a landmark-based alternative. Preserving these distinctions provides an accurate academic account and a clear foundation for reproducible follow-up work.